

Session 2 : CRUD 1

```
show databases;
```

```
create database school;
```

```
use school;
```

```
-- Create batches and students table:
```

```
create table batches(  
  batch_id int PRIMARY KEY,  
  batch_name varchar(50) not null);
```

```
create table students(  
  student_id int auto_increment PRIMARY KEY,  
  first_name varchar(50) not null,  
  last_name varchar(50) not null,  
  batch_id int,  
  FOREIGN KEY(batch_id) references batches(batch_id) ON DELETE CASCADE ON  
UPDATE CASCADE  
);
```

```
-- Inserting dummy data in both tables
```

```
insert into batches(batch_id, batch_name) values  
(1, 'Batch A'),  
(2, 'Batch B'),  
(3, 'Batch C');
```

```
insert into students(first_name, last_name, batch_id) values  
( 'John', 'Doe', 1),
```

```
('Jane', 'Doe', 1),
('Sony', 'Sonam', 2),
('Jenny', 'Smith', 3),
('Jack', 'Sparrow', 2);
```

```
delete from batches
where batch_id = 1;
```

```
-- create table
```

```
br
```

```
CREATE TABLE students (
    id INT AUTO_INCREMENT,
    firstName VARCHAR(50) NOT NULL,
    lastName VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    dateOfBirth DATE NOT NULL,
    enrollmentDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    psp DECIMAL(3, 2) CHECK (psp BETWEEN 0.00 AND 100.00),
    batchId INT,
    isActive BOOLEAN DEFAULT TRUE,
    PRIMARY KEY (id)
);

INSERT INTO students (firstName, lastName, email, dateOfBirth, psp,
batchId)
VALUES
('Chris', 'Green', 'chris.green@example.com', '2000-12-01', 67.45, 104),
('Sophia', 'Black', 'sophia.black@example.com', '2001-06-15', 95.60, 105),
('Michael', 'Blue', 'michael.blue@example.com', '1998-02-28', 80.00, 101);
```

Session 3 : CRUD 2

```
-- CRUD 2:

create database scaler;

use scaler;

-- Create batches and students table:
create table students(
    student_id int auto_increment PRIMARY KEY,
    first_name varchar(50) not null,
    last_name varchar(50),
    isActive BOOLEAN DEFAULT TRUE
);

-- inserting data without columns
-- Value count has to match column count
INSERT INTO STUDENTS
VALUES (default, 'Virat', null, default);

-- inserting data with columns
INSERT INTO STUDENTS (first_name, last_name)
VALUES ('Rohit', 'Sharma');

select first_name, last_name from students;
```

```
-- FOREIGN KEY CONCEPT
```

```
use scaler;
```

```
create table departments (  
    deptId INT auto_increment PRIMARY KEY,  
    deptName VARCHAR(50) NOT NULL  
);
```

```
create table employees (  
    employeeID INT AUTO_INCREMENT,  
    firstName VARCHAR(50) NOT NULL,  
    lastName VARCHAR(50), deptId INT,  
    PRIMARY KEY (employeeID),  
    FOREIGN KEY (deptId) references departments(deptId)  
);
```

```
drop table employees;
```

```
INSERT into departments (deptName)  
values ("HR"),  
("TECH"),  
("PRODUCT"),  
("MARKETING");
```

```
select * from departments;
```

```
INSERT INTO employees (firstName , lastName, deptID)  
values ("Virat" , "Kohli" , 1),  
("Rohit" , "Sharma" , 2),  
("Jasprit", "Bumrah" , 4);
```

```
select * from employees;
```

```
delete from departments where deptID = 4;

---- SELECT CONCEPT

use sakila;

select title as movie_name
from film;

select 'Hello World' as output;

select rental_duration, rating from film;

select distinct rental_duration, rating from film;

-- (1,2,"a",4,6)
-- (1,2,"a",5,6)

-- where

select * from film;

select * from film
where rating = 'G';

-- if(rating=='G') arr.add(x)

-- movies with rating 'G' OR 'PG'

-- logical operators in SQL
-- AND, OR, NOT
```

```
select * from film
where rating = 'G' or rating = 'PG';
```

```
-- all movies except rating 'G'
```

```
select * from film
where NOT (rating = 'G');
```

```
select * from film
where rating = 'G' AND rental_duration=6 OR rental_duration=7;
```

```
-- preference NOT > AND > OR
```

```
-- give me all the movies with rating either G, PG, NC-17, PG-13
```

```
select * from film
where rating = 'G' OR rating = 'PG-13' or rating = 'NC-17' or rating = 'PG';
```

```
select * from film
where rating IN ('G', 'PG', 'NC-17');
```

```
-- limit and offset
```

```
-- limit - no of records, offset - after which filmposition
```

```
select * from film
limit 18446744073709551615 offset 9;
```

```
-- ORDER BY
```

```
select * from film
order by replacement_cost;
```

```
select * from film
order by replacement_cost DESC;
```

```
-- By default PK is the tie breaker
-- default order is asc

-- where is executed first then order by
select * from film
where rental_duration > 5
order by title;

-- order by with distinct
SELECT DISTINCT title FROM film ORDER BY release_year;

-- LIKE operator
-- by default is case insensitive
select * from film
where binary title = 'Indian love';

-- concept of NULL
select NULL=NULL;

-- IS NULL
select * from film
where title IS NULL;

select * from film
where title IS NOT NULL;

-- update
-- update rental duration of films release in 2006 to 10
update film set rental_duration = 10
where release_year = 2006 and title IS NOT NULL;

SET SQL_SAFE_UPDATES = 1; -- update and delete operations
```

```
-- DROP VS DELETE VS TRUNCATE
```

Session 3 : Joins

```
-- Inner Joins:
```

```
-- Join
```

```
-- Get all the students along with their batch names
```

```
select s.name as student_name, b.name as batch_name
```

```
from students as s
```

```
cross join batches as b
```

```
on s.batch_id = b.batch_id;
```

```
-- Let's say for every film, we want to print its name and the language. How  
can we do that?
```

```
use sakila;
```

```
SELECT f.title, l.name
```

```
FROM film f
```

```
JOIN language l
```

```
ON f.language_id = l.language_id;
```

-- Left Join

-- -- Get all the students along with the batch name. Get unassigned students as well?

use academy;

```
select s.name, s.psp, b.name as batch_name
from students s
left join batches b
on s.batch_id = b.batch_id
where s.psp > 85;
```

-- Right Join

-- Get all the batches along with students assigned to them.

-- Include batches without students as well.

```
select *
from students s
right join batches b
on s.batch_id = b.batch_id;
```

```
select *
from batches b
left join students s
on s.batch_id = b.batch_id;
```

```
-- self join
-- get employees and their manager names
```

```
select e1.emp_name as emp_name, e2.emp_name as manager_name
from
employees e1
left join employees e2
on e1.manager_id = e2.emp_id
where e2.emp_name = 'Tony';
```

```
-- Joining multiple tables
-- Get name of actors along with the name of films they acted in?
```

```
use sakila;
select f.title, a.first_name, a.last_name
from film f
join film_actor fa
on f.film_id = fa.film_id
join actor a
on a.actor_id = fa.actor_id;
```

```
-- -- Get data of all the actors who performed in 'ACADEMY DINOSAUR'
ordered by actor's first_name:
```

```
select f.title, a.first_name, a.last_name
from film f
join film_actor fa
```

```
on f.film_id = fa.film_id
join actor a
on a.actor_id = fa.actor_id
where f.title = 'ACADEMY DINOSAUR'
order by a.first_name;
```

```
use academy;
```

```
-- create employees
CREATE TABLE employees (
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50) NOT NULL,
  manager_id INT,
  FOREIGN KEY (manager_id) REFERENCES employees(emp_id)
);
```

Session 5 : Aggregates and Group by

```
use academy;
```

```
-- How many students have been assigned to some batch?
select count(batch_id)
from students;
```

```
-- How many students have been assigned to some batch?
select count(bsp)
```

```
from students;
```

```
-- Get the count of unique batches assigned to students?
```

```
select count(distinct(batch_id))
```

```
from students;
```

```
-- not correct
```

```
select distinct(count(batch_id))
```

```
from students;
```

```
-- Get total number of student entries in students table?
```

```
select count(id)
```

```
from students;
```

```
-- error
```

```
-- Never put non agg and agg columns together in select
```

```
select count(id), name
```

```
from students;
```

```
-- no of rows in the table/ total no of student
```

```
select count(1)
```

```
from students;
```

```
select count(*)
```

```
from students;
```

```
-- Other aggregate functions:
```

```
-- get avg psp of all student
```

```
select avg(psp), sum(psp), max(psp), min(psp)
```

```
from students;
```

```
-- wrng avg
```

```
select sum(batch_id)/count(batch_id)
```

```
from students;
```

```
select sum(batch_id)/count(*)
```

```
from students;
```

```
select avg(batch_id), batch_id  
from students;
```

```
-- Get avg(psp) of learners along with their batch_id and batch_name?
```

```
select avg(psp) as average, s.batch_id, b.name as batch_name  
from students s  
join batches b  
on s.batch_id = b.batch_id  
group by s.batch_id;
```

```
-- Get all the batches with their avg(psp) include only those batches having avg psp > 85
```

```
select avg(psp) as psp, batch_id  
from students  
group by batch_id  
having psp > 85;
```

```
select avg(psp) as avg_psp, batch_id  
from students  
where batch_id <> 2  
group by batch_id  
having avg_psp > 85 and name<> 'Virat';
```

```
-- H.W. Find rental_duration which is rental_duration of more than 200 movies
```

```
-- Cross join
```

```
select *  
from students s  
join batches b  
on s.batch_id <> b.batch_id;
```

```
select round(6.783993,2)
```

Session 6 : Aggregates and Group by

working session

– Project Workload Analysis by Country

```
select p.project_name, e.country, count(distinct e.emp_id) as employee_count
from project p
join assignment a
on p.project_id = a.project_id
join employee e
on e.emp_id = a.emp_id
where p.project_status = 'Active'
group by p.project_name, e.country
order by p.project_name, e.country;
```

– Average Skill Proficiency by Department

```
select d.dept_name, s.skill_name, round(avg(es.proficiency_level),2) as avg_proficiency
from department d
join employee e
on d.department_id = e.department_id
join emp_skill es
on es.emp_id = e.emp_id
join skill s
on s.skill_id = es.skill_id
where es.verified_flag = 'Y'
group by s.skill_name, d.dept_name
having count(distinct e.emp_id) >= 3
order by d.dept_name, s.skill_name;
```

–Overloaded Employees with Skill and Assignment Details

```
select e.emp_name,  
       d.dept_name,  
       e.country,  
       SUM(a.allocation_pct) AS total_allocation,  
       GROUP_CONCAT(DISTINCT s.skill_name) AS skills  
from employee e  
join assignment a on e.emp_id = a.emp_id  
join project p on p.project_id = a.project_id and p.project_status = 'Active'  
join department d on e.department_id = d.department_id  
left join emp_skill es on e.emp_id = es.emp_id AND es.verified_at != '1900-01-01'  
left join skill s on s.skill_id = es.skill_id  
group by e.emp_id, e.emp_name, d.dept_name, e.country  
having total_allocation > 100  
ORDER BY total_allocation DESC;
```

– Students in Departments

```
select d.dept_name, count(s.student_id) as student_number  
from student s  
right join department d on s.dept_id = d.dept_id  
group by d.dept_name  
order by student_number DESC;
```

– Joined first

```
select a.employee_id, a.first_name, a.last_name  
from employees a  
join employees b on a.manager_id = b.employee_id  
where a.hire_date < b.hire_date  
order by a.employee_id
```

–Students and Examinations

```
select s.student_id, s.student_name, st.subject_name, count(e.subject_name) as
attended_exams
from students s
cross join subjects st
left join examinations e on s.student_id = e.student_id and e.subject_name =
st.subject_name
group by s.student_id, s.student_name, st.subject_name
order by s.student_id, st.subject_name;
```

Session 7 : Subqueries and Views

```
use academy;
```

```
-- Subqueries and Views:
```

```
-- Question 1: Get all the students having psp > psp of student with
student_id = 2
```

```
-- step 1
```

```
select psp
from students
where id = 2;
```

```
-- x = 90
```

-- Step 2: Get all students with psp > x

```
select *  
from students  
where psp > (  
select psp  
from students  
where id = 1  
);
```

-- Question 2: Find data of all students having psp > min(psp) of b_id 2.

-- Step 1: Get min psp of batch 3

```
select min(psp)  
from students  
where batch_id = 3;
```

-- x= 70

-- Step 2: Get all studentst with psp > x

```
select *  
from students  
where psp > (  
select min(psp)  
from students  
where batch_id = 3  
);
```

-- Question 3: Find all the years where avg(rental_rate) > global
avg(rental_rate)
use sakila;

-- Step 1: Find global avg

```
select avg(rental_rate)
```

```
from film;
```

```
-- x = 2.92
```

```
-- Step 2: avg per year
```

```
select avg(rental_rate), release_year  
from film
```

```
group by release_year
```

```
having avg(rental_rate) >= (select avg(rental_rate)  
from film);
```

```
-- Question: Find data of all students where psp > (min(bsp) among  
avg(bsp) of every batch)
```

```
-- Step 1: Get avg of each batch
```

```
use academy;
```

```
select avg(bsp)
```

```
from students
```

```
group by batch_id;
```

```
-- x
```

```
-- Step 2, get min y
```

```
select min(group_bsp)
```

```
from (select avg(bsp) as group_bsp
```

```
from students
```

```
group by batch_id) as t;
```

```
-- Step 3: get all students with bsp > y
```

```
select * from students
```

```
where bsp > (
```

```
select min(group_bsp)
```

```
from (select avg(bsp) as group_bsp
```

```
from students
group by batch_id) as t
);
```

-- ALL and ANY

-- Question: Find data of all learners where psp >= min(psp) of every batch

-- Step 1: Get min batch wise

```
select min(psp)
from students
group by batch_id;
```

-- x : [80,78,90,...]

-- Step 2: student with psp >= All(x)

```
select * from
students
where psp >= All(select min(psp)
from students
group by batch_id);
```

-- Question: Find data of all learners where psp >= min(psp) of any of the batch

```
select * from
students
where psp >= ANY(select min(psp)
from students
group by batch_id);
```

-- Co-related sub-queries

-- Question: Find all the students where psp > avg(psp) of their own batch

-- Step 1: Get avg psp of batch 4

```
select avg(bsp)  
from students  
where batch_id = 4;
```

-- get all the students with psp > avg(bsp) of batch 4

```
select *  
from students s  
where bsp > (  
select avg(bsp)  
from students  
where batch_id = s.batch_id  
);
```

-- Exists

use sakila;

-- Find all actors who have acted in atleast one movie.

-- check if actor with id 10 has acted in a film or not

```
select *  
from film_actor  
where actor_id = 10;
```

-- get the details of the actor with id 10 only if he has acted in a movie

```
select * from  
actor a  
where exists (  
select *  
from film_actor  
where actor_id = a.actor_id);
```

-- Views

```
select f.title, concat(a.first_name, ' ', a.last_name) full_name
from film f
join film_actor fa
on f.film_id = fa.film_id
join actor a
on a.actor_id = fa.actor_id;
```

-- create view

```
create or replace view actor_film_info as
select f.title, concat(a.first_name, ' ', a.last_name) full_name
from film f
join film_actor fa
on f.film_id = fa.film_id
join actor a
on a.actor_id = fa.actor_id;
```

```
select * from actor_film_info;
```

Session 7 : Indexing

```
SELECT * FROM sakila.film;
```

```
-- -> Index lookup on film using idx_title (title='AFRICAN EGG') (cost=0.35  
rows=1) (actual time=0.0248..0.0273 rows=1 loops=1)
```

```
explain analyze select * from  
film  
WHERE title = 'AFRICAN EGG';
```

```
drop index idx_title  
on film;
```

```
-- without index  
-- -> Table scan on film (cost=103 rows=1000) (actual time=0.0345..0.707  
rows=1000 loops=1)  
explain analyze select * from  
film  
WHERE title = 'AFRICAN EGG';
```

```
-- 1 character  
create index idx_title  
on film(title(1));
```

```
-- -> Index lookup on film using idx_title (title='AFRICAN EGG') (cost=13.6  
rows=46) (actual time=0.0217..0.088 rows=46 loops=1)
```

```
explain analyze select * from  
film  
WHERE title = 'AFRICAN EGG';
```

```
-- 2 character  
create index idx_title  
on film(title(2));
```

```
-- -> Index lookup on film using idx_title (title='AFRICAN EGG') (cost=0.7  
rows=2) (actual time=0.0206..0.0249 rows=2 loops=1)  
explain analyze select * from  
film  
WHERE title = 'AFRICAN EGG';
```

Session 10 : CTE and Window functions

```
-- CTE
```

```
use sakila;
```

```
-- Question: Customers who spent more than $100 using subquery:
```

```
-- total spends of every customer
```

```
select * from
```

```
(select c.customer_id, c.first_name, sum(p.amount) as total_sum
```

```
from customer c
```

```
join payment p
```

```
on c.customer_id = p.customer_id
```

```
group by c.customer_id) as customer_spending
```

```
where total_sum > 100;
```

```
-- Let's write same query using a CTE:
```

```
with customer_spending as (
```

```
select c.customer_id, c.first_name, sum(p.amount) as total_sum
```

```
from customer c
```

```
join payment p
```

```
on c.customer_id = p.customer_id
```

```
group by c.customer_id
)
select * from
customer_spending
where total_sum > 200;
```

```
select * from
customer_spending
where total_sum > 200;
```

```
-- Using multiple CTEs in s query:
-- To find the top 5 customers in the Sakila database based on how much
total payment
-- their name,
-- number of rentals they made,
-- total amount paid, and
-- average payment amount per customer.
```

```
-- Step 1: Count total rentals for each customer
select customer_id, count(*) as total_rentals
from rental
group by customer_id;
```

```
-- Step 2: Calculate total and average payment per customer:
select customer_id, sum(amount) as total_payment, avg(amount) as
avg_payment
from payment
group by customer_id;
```

```
-- create 2 CTEs
with customer_rental as (
```

```
select customer_id, count(*) as total_rentals
from rental
group by customer_id
),
customer_payments as (
select customer_id, sum(amount) as total_payment, avg(amount) as
avg_payment
from payment
group by customer_id
)
select c.first_name, cr.total_rentals, cp.total_payment, cp.avg_payment
from customer c
join customer_rental cr on c.customer_id = cr.customer_id
join customer_payments cp on c.customer_id = cp.customer_id
order by cp.total_payment desc
limit 5;
```

```
-- WINDOW function
use academy;
```

```
select name, psp, max(psp) over() as max_marks
from students;
```

```
-- Get avg(rental_duration) with each film:
use sakila;
```

```
select *, avg(rental_duration) over() as rental_duration
from film;
```

```
-- Partition by:
```

```
use academy;
```

```
select *, max(salary) over(PARTITION BY department, salary) as  
max_salary  
from employees;
```

```
-- Get total rentals along with all the rental data without collapsing the rows:  
-- Should contain global total rental and per customer rental as well
```

```
use sakila;  
select rental_id, customer_id,  
count(*) over(partition by customer_id) as total_rental_per_cust,  
count(*) over() as total_rental  
from rental  
order by total_rental_per_cust desc;
```

```
-- over() with ranking  
use academy;  
-- rank()
```

```
select *,  
RANK() over(order by salary desc) as normal_rank,  
DENSE_RANK() over(order by salary desc) as denserank,  
ROW_NUMBER() over(partition by department order by salary desc) as  
rowNumber  
from employees;
```

```
-- Rank, Dense_rank, row_number:  
-- Rank the movies by number of rentals:  
use sakila;
```

```
select  
f.film_id,  
f.title,  
count(r.rental_id) as total_rental,  
RANK() over(order by count(r.rental_id)) as normal_rank,
```

```
DENSE_RANK() over(order by count(r.rental_id)) as denseRank,  
ROW_NUMBER() over(order by count(r.rental_id)) as rowNumber  
from film f  
join inventory i  
on f.film_id = i.film_id  
join rental r  
on i.inventory_id = r.inventory_id  
group by f.film_id, f.title;
```